

Experiment - 2

Goal: Write a program in python to implement Breadth First Search (BFS) algorithm for a given graph.

Code:

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    # To keep track of visited nodes
    # Initialize queue with starting node

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        # Visit all adjacent nodes
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Example graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

# Call BFS starting from node 'A'
bfs(graph, 'A')
```

Output:

A	B	C	D	E	F
---	---	---	---	---	---

Experiment - 3

Goal: Write a program in python to implement Depth First Search (DFS) algorithm for a given graph.

Code:

```
from collections import deque
```

```
def bfs(graph, node, visited = None):  
    if visited is None:  
        visited = set()  
    visited.add(node)  
    print(node, end = " ")  
  
    while queue:  
        node = queue.popleft()  
        print(node, end=" ")  
  
    for neighbor in graph[node]:  
        if neighbor not in visited:  
            dfs(graph, neighbour, visited)
```

Example graph (Adjacency List)

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D', 'E'],  
    'C': ['A', 'F'],  
    'D': ['B'],  
    'E': ['B', 'F'],  
    'F': ['C', 'E']  
}
```

Call DFS starting from node 'A'

```
bfs(graph, 'A')
```

Output:

```
A B D E F C
```

Experiment - 4

Goal: Write a program in python to implement Best First Search algorithm for a given graph.

Code:

```
import heapq
def best_first_search(graph, start, goal, heuristic):
    visited = set()
    priority_queue = []
    heapq.heappush(priority_queue, (heuristic[start], start))
    visited.add(start)
    while priority_queue:
        h_value, current = heapq.heappop(priority_queue)
        print(current, end=" ")
        if current == goal:
            print("\nGoal reached!")
            return
        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                heapq.heappush(priority_queue, (heuristic[neighbor], neighbor))
    print("\nGoal not reachable.")

# Example graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

# Heuristic values (smaller value = closer to goal)
heuristic = {
    'A': 5,
    'B': 3,
    'C': 4,
    'D': 2,
    'E': 6,
    'F': 1
}

# Start and Goal
best_first_search(graph, 'A', 'F', heuristic)
```

Output:

```
A B D C F
Goal reached!
```

Experiment - 5

Goal: Write a program in python to implement A* Search Algorithm.

Code:

```
import heapq
def a_star_search(graph, start, goal, heuristic):
    visited = set()
    priority_queue = []
    heapq.heappush(priority_queue, (heuristic[start], 0, start, [start]))
    visited.add(start)
    while priority_queue:
        f_value, g_value, current, path = heapq.heappop(priority_queue)
        if current in visited:
            continue
        print(current, end=" ")
        visited.add(current)
        if current == goal:
            print("\nGoal reached!")
            print("Path: ", "->".join(path))
            return
        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                heapq.heappush(priority_queue, (f_n, g_n, neighbor, path + [neighbour]))
    print("\nGoal not reachable.")
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1), ('E', 5)],
    'C': [('F', 2)],
    'D': [],
    'E': [],
    'F': []
}
heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 0,
    'E': 2,
    'F': 1
}
a_star_search(graph, 'A', 'D', heuristic)
```

Output:

```
A B D
Goal reached!
Path: A -> B -> D
```

Experiment - 6

Goal: Write a program in python to implement Travelling Salesman Problem.

Code:

```
import sys
def tsp(graph, start):
    n = len(graph)
    visited = [False] * n
    visited[start] = True
    min_cost = sys.maxsize
    best_path = [ ]
    def solve(curr, count, cost, path):
        nonlocal min_cost, best_path
        if count == n and graph[curr][start] > 0:
            total_cost = cost + graph[curr][start]
            if total_cost < min_cost:
                min_cost = total_cost
                best_path = path + [start]
        return
    for i in range(n):
        if not visited[i] and graph[curr][i] > 0:
            visited[i] = True
            solve(i, count + 1, cost + graph[curr][i], path + [i])
            visited[i] = False
    solve(start, 1, 0, [start])
    print("Minimum Cost:", min_cost)
    print("Path:", " -> ".join(map(str, best_path)))

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
tsp(graph, 0)
```

Output:

```
Minimum Cost: 80
Path: 0 -> 1 -> 3 -> 2 -> 0
```

Experiment - 7

Goal: Write a program in python to implement Hill Climbing and Steepest Ascent Hill Climbing Algorithm.

Code:

```
def hill_climbing(graph, start, heuristic):
    current = start
    path = [current]
    while True:
        neighbors = graph[current]
        next_node = None
        for neighbor in neighbors:
            if heuristic[neighbor] < heuristic[current]:
                next_node = neighbor
                break
        if next_node is None:
            break
        current = next_node
        path.append(current)
    print("Hill Climbing Path:", " -> ".join(path))

def steepest_ascent_hill_climbing(graph, start, heuristic):
    current = start
    path = [current]
    while True:
        neighbors = graph[current]
        best_neighbor = current
        for neighbor in neighbors:
            if heuristic[neighbor] < heuristic[best_neighbor]:
                best_neighbor = neighbor
        if best_neighbor == current:
            break
        current = best_neighbor
        path.append(current)
    print("Steepest Ascent Path:", " -> ".join(path))

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

heuristic = {
    'A': 6,
```

```
'B': 4,  
'C': 5,  
'D': 1,  
'E': 2,  
'F': 3  
}
```

```
hill_climbing(graph, 'A', heuristic)
```

```
steepest_ascent_hill_climbing(graph, 'A', heuristic)
```

Output:

```
Hill Climbing Path: A -> B -> D
```

```
Steepest Ascent Path: A -> B -> D
```

Experiment - 8

Goal: Write a program in python to load data in google colab notebook from CSV file.

Code:

```
import pandas as pd
from google.colab import files

uploaded = files.upload()

for file_name in uploaded.keys():
    df = pd.read_csv(file_name)
    print("File Loaded:", file_name)
    print(df.head())
```

Output:

```
Choose Files
sample_data.csv
sample_data.csv(application/vnd.ms-excel) - 1024 bytes, last modified: 02/04/2026
Saving sample_data.csv to sample_data (1).csv
```

```
File Loaded: sample_data (1).csv
```

	Name	Age	Marks
0	Aman	20	85
1	Riya	21	90
2	Karan	19	78
3	Neha	22	88
4	Ravi	20	92

Experiment - 9

Goal: Write a program in python to implement various Panda's command on Titanic dataset.

Code:

```
import pandas as pd
df = pd.read_csv("/mnt/data/Titanic-Dataset.csv")
print("First 5 Rows:")
print(df.head())
print("\nLast 5 Rows:")
print(df.tail())
print("\nDataset Info:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())
print("\nColumn Names:")
print(df.columns)
print("\nSelect Specific Columns:")
print(df[['Name', 'Age', 'Fare']].head())
print("\nFilter: Age > 30")
print(df[df['Age'] > 30].head())
print("\nFilter: Survived = 1")
print(df[df['Survived'] == 1].head())
print("\nGroup By Pclass:")
print(df.groupby('Pclass').mean(numeric_only=True))
print("\nMissing Values:")
print(df.isnull().sum())
print("\nFill Missing Age:")
df['Age'] = df['Age'].fillna(df['Age'].mean())
print(df['Age'].head())
print("\nSort by Fare:")
print(df.sort_values(by='Fare', ascending=False).head())
print("\nUnique Embarked:")
print(df['Embarked'].unique())
print("\nSurvival Count:")
print(df['Survived'].value_counts())
```

Output:

```
First 5 Rows:
 PassengerId  Survived  Pclass
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

   Name                               Sex  Age  SibSp
0  Braund, Mr. Owen Harris             male  22.0    1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0    1
2    Heikkinen, Miss. Laina             female  26.0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)   female  35.0    1
4    Allen, Mr. William Henry             male  35.0    0

   Parch  Ticket   Fare  Cabin  Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282  7.9250   NaN      S
3      0   113803  53.1000  C123      S
4      0   373450   8.0500   NaN      S
```

```
Last 5 Rows:
 PassengerId  Survived  Pclass                               Name
886          887         0       2          Montvila, Rev. Juozas
887          888         1       1    Graham, Miss. Margaret Edith
888          889         0       3  Johnston, Miss. Catherine Helen "Carrie"
889          890         1       1          Behr, Mr. Karl Howell
890          891         0       3        Dooley, Mr. Patrick

   Sex  Age  SibSp  Parch  Ticket   Fare  Cabin  Embarked
886  male  27.0    0     0   211536  13.00   NaN      S
887  female  19.0    0     0   112053  30.00   B42      S
888  female  NaN    1     2  W./C. 6607  23.45   NaN      S
889  male  26.0    0     0   111369  30.00  C148      C
890  male  32.0    0     0   370376   7.75   NaN      Q
```

Dataset Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object
4	Sex	891 non-null	object
5	Age	714 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	object
9	Fare	891 non-null	float64
10	Cabin	204 non-null	object
11	Embarked	889 non-null	object

dtypes: float64(2), int64(5), object(5)

memory usage: 83.7+ KB

None

Statistical Summary:

	PassengerId	Survived	Pclass	Age	SibSp
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Column Names:

Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'], dtype='object')

Select Specific Columns:

	Name	Age	Fare
0	Braund, Mr. Owen Harris	22.0	7.2500
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	71.2833
2	Heikkinen, Miss. Laina	26.0	7.9250
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	53.1000
4	Allen, Mr. William Henry	35.0	8.0500

Filter: Age > 30

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0

	Parch	Ticket	Fare	Cabin	Embarked
1	0	PC 17599	71.2833	C85	C
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
6	0	17463	51.8625	E46	S
11	0	113783	26.5500	C103	S

Filter: Survived = 1

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1

	Parch	Ticket	Fare	Cabin	Embarked
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
8	2	347742	11.1333	NaN	S
9	0	237736	30.0708	NaN	C

Group By Pclass:

	PassengerId	Survived	Age	SibSp	Parch	Fare
Pclass						
1	461.597222	0.629630	38.233441	0.416667	0.356481	84.154687
2	445.956522	0.472826	29.877630	0.402174	0.380435	20.662183
3	439.154786	0.242363	25.140620	0.615071	0.393075	13.675550

Fill Missing Age:

0	22.0
1	38.0
2	26.0
3	35.0
4	35.0

Name: Age, dtype: float64

Survival Count:

Survived	
0	549
1	342

Name: count, dtype: int64

Missing Values:

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

Sort by Fare:

	PassengerId	Survived	Pclass	Name
258	259	1	1	Ward, Miss. Anna
737	738	1	1	Lesurer, Mr. Gustave J
679	680	1	1	Cardeza, Mr. Thomas Drake Martinez
88	89	1	1	Fortune, Miss. Mabel Helen
27	28	0	1	Fortune, Mr. Charles Alexander

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
258	female	35.0	0	0	PC 17755	512.3292	NaN	C
737	male	35.0	0	0	PC 17755	512.3292	B101	C
679	male	36.0	0	1	PC 17755	512.3292	B51 B53 B55	C
88	female	23.0	3	2	19950	263.0000	C23 C25 C27	S
27	male	19.0	3	2	19950	263.0000	C23 C25 C27	S

Unique Embarked:

['S' 'C' 'Q' nan]

Experiment - 10

Goal: Write a program to integrate Python and SQL.

Code:

```
import sqlite3
conn = sqlite3.connect("student.db")
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS students(id INTEGER PRIMARY KEY, name TEXT, age INT, marks REAL)")
cur.executemany("INSERT INTO students(name, age, marks) VALUES(?,?,?)",
                [('Aman',20,85), ('Riya',21,90), ('Karan',19,78)])
conn.commit()
print(cur.execute("SELECT * FROM students").fetchall())
cur.execute("UPDATE students SET marks=88 WHERE name='Aman'")
cur.execute("DELETE FROM students WHERE name='Karan'")
conn.commit()
print(cur.execute("SELECT * FROM students").fetchall())
conn.close()
```

Output:

```
[(1, 'Aman', 20, 85.0), (2, 'Riya', 21, 90.0), (3, 'Karan', 19, 78.0)]

[(1, 'Aman', 20, 88.0), (2, 'Riya', 21, 90.0)]
```

Experiment - 11

Goal: Write a program in python to build a Basic Linear Regression Model.

Code:

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])
model = LinearRegression()
model.fit(X, y)
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
y_pred = model.predict(X)
print("Predictions:", y_pred)
new_value = np.array([[6]])
print("Prediction for 6:", model.predict(new_value)[0])
```

Output:

```
Coefficient: 2.0
Intercept: 0.0
Predictions: [ 2.  4.  6.  8. 10.]
Prediction for 6: 12.0
```

Experiment - 12

Goal: Write a program in python to implement Data Visualisation techniques such as plotting bar-graph, histogram, pie-charts using Matplotlib.

Code:

```
import matplotlib.pyplot as plt
categories = ['A', 'B', 'C', 'D']
values = [10, 20, 15, 25]
plt.bar(categories, values)
plt.title("Bar Graph")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()
data = [12, 15, 13, 17, 18, 19, 21, 22, 25, 27]
plt.hist(data, bins=5)
plt.title("Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
labels = ['Python', 'Java', 'C++', 'Others']
sizes = [40, 25, 20, 15]
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```

Output:

